

Si parte da un nome simbolico, come `www.google.com`, che è ciò che tipicamente utilizza un'applicazione. Questo nome non è direttamente utilizzabile per l'instradamento dei pacchetti nella rete, quindi deve essere tradotto in un indirizzo IP. Questa traduzione è effettuata dal sistema DNS (Domain Name System), che associa a ciascun nome un indirizzo numerico, ad esempio `142.250.151.147`.

L'indirizzo IP identifica in modo univoco una macchina all'interno della rete globale.

Tuttavia, l'indirizzo IP da solo non è sufficiente a individuare il destinatario finale della comunicazione, perché su una stessa macchina possono essere in esecuzione più processi che offrono servizi diversi. Per distinguere questi processi si utilizza il concetto di porta.

La porta è un numero che identifica un servizio specifico all'interno della macchina. Ad esempio, il servizio web HTTP è convenzionalmente associato alla porta 80.

Dal punto di vista del processo, l'accesso alla rete non avviene direttamente tramite IP e porta, ma attraverso un'astrazione fornita dal sistema operativo: il socket. Il socket è rappresentato nel processo tramite un file descriptor (fd), cioè un intero che identifica una risorsa aperta.

I file descriptor sono usati in generale per rappresentare file, dispositivi e anche connessioni di rete. Nel caso dei socket, il file descriptor diventa il punto attraverso cui il processo legge e scrive dati verso la rete, utilizzando le stesse primitive di I/O (come `read` e `write`) usate per i file.

Sul lato client, il processo crea un socket (quindi ottiene un file descriptor) e, dopo aver risolto il nome in indirizzo IP e individuato la porta del servizio, effettua una richiesta di connessione (`connect`) verso la coppia (IP, porta) del server.

Il sistema operativo assegna automaticamente al client una porta locale, detta porta effimera, che serve a distinguere quella specifica comunicazione.

La comunicazione effettiva avviene quindi tra due estremi ben definiti: da un lato la coppia (IP client, porta client), dall'altro la coppia (IP server, porta server). I due socket, identificati nei rispettivi processi dai loro file descriptor, costituiscono gli endpoint della connessione.

Tutti i dati scambiati transitano attraverso questi file descriptor, che rappresentano il collegamento concreto tra il processo e la rete.

```
l'indirizzo IP individua univocamente la macchina a livello globale,  
  
la porta individua univocamente il processo all'interno della macchina,  
  
il file descriptor rappresenta univocamente , all'interno del processo,  
il canale operativo attraverso cui avviene la comunicazione.
```

Nell'interfaccia dei socket, il polimorfismo non è ottenuto con classi o metodi virtuali, ma con una tecnica tipicamente C: si definisce una struttura molto generale, `struct sockaddr`, che funge da tipo astratto comune, e poi si introducono strutture specializzate per le diverse famiglie di indirizzi, come `struct sockaddr_in` per `AF_INET`, `struct sockaddr_in6` per `AF_INET6`, `struct sockaddr_un` per i socket Unix domain

L'idea è che le system call, come connect, ricevano sempre un puntatore al tipo generico, ma in realtà quel puntatore indirizza memoria che contiene una struttura concreta della famiglia corretta.

La struttura generica sockaddr è deliberatamente minimale. Contiene essenzialmente due parti:

1. un campo che indica la famiglia di indirizzi, sa_family.
2. un'area di byte non interpretata, sa_data, pensata come contenitore generico.

Non è una struttura che l'applicazione usa davvero per costruire un indirizzo IPv4 o IPv6; serve piuttosto come interfaccia uniforme. La struttura concreta, ad esempio sockaddr_in, contiene invece i campi veri che servono per IPv4: la famiglia sin_family, il numero di porta sin_port l'indirizzo sin_addr.

Queste strutture sono progettate in modo da avere all'inizio un campo di famiglia compatibile concettualmente con quello della sockaddr generica, così che il kernel possa leggere subito da lì il tipo di indirizzo che gli viene passato.

Il meccanismo è questo: il programma alloca e riempie una struttura specifica, per esempio una struct sockaddr_in, e poi passa il suo indirizzo a connect, ma dopo averlo convertito in un puntatore a struct sockaddr. Quindi non si crea davvero una "specializzazione" a runtime nel senso dei linguaggi a oggetti; accade piuttosto il contrario: si crea direttamente l'oggetto concreto e lo si presenta alla funzione con il tipo più generale richiesto dall'interfaccia. Il polimorfismo sta nel fatto che la stessa funzione connect accetta un puntatore formalmente uniforme, ma poi decide come interpretare la memoria in base al contenuto del campo sa_family e anche in base al tipo di socket già creato.

Per esempio, un codice tipico è questo: int socket(int domain, int type, int protocol)

```
int fd = socket(AF_INET, SOCK_STREAM, 0);
```

```
struct sockaddr_in addr{ sa_family_t sin_family; famiglia di indirizzi in_port_t sin_port; numero di porta struct in_addr sin_addr; indirizzo IP }
```

```
addr.sin_family = AF_INET;
```

```
addr.sin_port = htons(80);
```

```
addr.sin_addr = ....
```

```
...
```

```
SYSTEM CALL CONNECT: connect(fd, (struct sockaddr *)&addr, sizeof(addr));
```

Qui addr è una sockaddr_in, quindi una struttura specializzata per IPv4. La funzione connect, però, ha prototipo:

```
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

e dunque pretende un const struct sockaddr *. Siccome in C un struct sockaddr_in * non è automaticamente convertibile a struct sockaddr *, è necessario il cast esplicito:

```
(struct sockaddr *)&addr
```

Quel cast non trasforma il contenuto della struttura; dice soltanto al compilatore di trattare quell'indirizzo come puntatore al tipo generico richiesto dall'interfaccia. La memoria resta quella di una `sockaddr_in`.

A questo punto sorge la domanda decisiva: come fa connect a capire che non deve trattare quell'oggetto come una `sockaddr` "vera", ma come una `sockaddr_in`? La risposta è che il kernel, quando riceve il puntatore dall'interfaccia di sistema, legge il campo iniziale della struttura, cioè la famiglia di indirizzi. Se trova `AF_INET`, sa che il buffer puntato da `addr` deve essere interpretato come una struttura `IPv4`; quindi si aspetta che in quella memoria, nella disposizione prevista per `sockaddr_in`, ci siano una porta a 16 bit in network byte order e un indirizzo `IPv4`.

Se invece il campo di famiglia contiene `AF_INET6`, la stessa funzione instraderà l'elaborazione verso il codice interno dedicato a `IPv6` e interpreterà il buffer come una `sockaddr_in6`.

Se trova `AF_UNIX`, lo interpreterà secondo il formato di `sockaddr_un`.

In altre parole, la dispatch polimorfica non avviene sul tipo statico del puntatore, perché dopo il cast quel tipo è sempre `struct sockaddr *`, ma sul contenuto della memoria puntata, in particolare sul discriminante `sa_family`.

Il terzo parametro di connect, cioè `addrlen`, è anch'esso fondamentale. Non basta infatti passare il puntatore generico: bisogna anche dire quanti byte della struttura concreta sono validi. Nel caso `IPv4` si passa tipicamente `sizeof(struct sockaddr_in)`,

nel caso `IPv6` `sizeof(struct sockaddr_in6)`.

Questo consente al kernel di sapere quanto grande è il blocco di memoria da leggere e di verificare che la dimensione sia coerente con la famiglia dichiarata. Quindi il comportamento dipende sia da `sa_family` sia dalla lunghezza fornita.

SYSTEM CALL `write()` utilizzata per scrivere dati in un FILE DESCRIPTOR da un buffer. Nel nostro caso permette di scrivere sul fd del socket del client dal buffer "request", scrivendola tutta in un colpo.

```
size_t write(int fd, const void *buf, size_t count);
```

- `int fd`: Il FILE DESCRIPTOR su cui si vuole scrivere i dati, nel nostro caso quello fornito dal socket del client.
- `const void *buf`: un puntatore al (alla prima cella da dove salvare di) buffer che contiene i dati da scrivere.
- `size_t count`: Il numero di byte da scrivere dal buffer al file descriptor.

- ritorna il numero di byte letti

ES: `write(s, request, strlen(request))` //scrivi `strlen(request)` byte (tutti i byte) del buffer `request` nel fd `s` del socket del client

SYSTEM CALL `read()` La system call `read()` è utilizzata per leggere dati da un FILE DESCRIPTOR in un buffer. Nel nostro caso permette di leggere dal fd del socket del client sul buffer "response", leggendo tutti i dati attualmente disponibili man mano che arrivano dati.

`size_t read(int fd, const void *buf, size_t count);`

- `int fd`: Il FILE DESCRIPTOR da cui si vuole leggere i dati, nel nostro caso quello fornito dal socket del client.
- `const void *buf`: un puntatore a (alla prima cella da dove salvare di) un buffer dove i dati letti saranno memorizzati.
- `size_t count`: Il numero massimo di byte da leggere dal file descriptor al buffer.
- ritorna il numero di byte letti

ES: `for (len = 0; (n = read(s, response+len, 100000 - len)) > 0; len+=n)` //finchè ciò che leggo dal fd del socket `s` del client è `>0`, leggi dal socket del client `100000 - len` byte, ovvero tutti i byte presenti in `s` non ancora letti e salvati in `response + len`. Poi incrementa `len` del numero di byte letti da `s` in quella iterazione, così da leggere e scrivere solo i byte rimanenti da `s` al buffer `response` all'iterazione successiva.